

and C# and Java collections. The output of the program is shown in Figure 2. You can debug any Vala code snippet through *gdb*. To generate an executable with debug symbols, compile your Vala code with the additional `-g --save-temps` switches, and then launch the executable with *gdb* to debug. The `--save-temps` switch enables line-by-line mapping of Vala code and C code in generated intermediate C files, to help you in debugging.

OOP with Vala

Let's move from procedural code to look at Vala's powerful support for object-oriented programming around various GNOME-provided objects. You deal with the various classes derived from GLib's *Object* class almost all the time, to take full advantage of GObject. You instantiate objects in Vala with the *new* keyword, like in C# and Java. Vala does automatic garbage-collection using reference counting on your objects. Vala doesn't provide multiple inheritance—your class can derive from one superclass, at most. You can define *properties* in a class, with *get* and *set* methods as accessors and mutators for the private class data. You can do run-time object type discovery—by using the *name* method of a *Type* reference (returned by the inherited *get_type* method of the object). Compile *testoop.vala* (source shown below) and run it to see these concepts in action.

```
class WObj : Object
{
    private int _ival;

    public int ival {
        get { return _ival; }
        set { _ival = value; _dval = 2; }
    }

    double dval;

    WObj(double dval) {
        _ival = 7;
        this.dval = dval;
    }

    public void showState() {
        stdout.printf("%s ival : %d, dval : %f\n", _ival, _dval);
    }
}
```

```
}

public static void main() {
    var wobj = new WObj(3.14);

    Type type = wobj.get_type();
    stdout.printf("%s wobj.get_type() : %s\n", type.name());
    wobj.showState();
    wobj._ival = 18;
    wobj.showState();
}
}
```

In the *set* mutator, *value* is the new value passed by calling code to the mutator.

Vala doesn't provide method overloading, so you can't declare and define multiple functions and constructors with the same name and different signatures. You can use the *named constructors* feature, which allows you to give different name additions to various constructors to overload them. You can also chain constructors. Vala also provides destructors, for very rare cases when you manually manage memory with pointers. Vala does a *null check* on function parameters and return values—like static checking to prevent runtime errors like dereferencing a null reference. If you want a parameter or return value to be able to have a null value, then use the *?* modifier after its name. When you work with multiple Vala source files, to create an executable, you only have to supply the file names to *valac*—it automatically figures out how to connect the different pieces of code to generate the executable.

Now we see a basic example of curses programming in Vala. This example requires the ncurses headers in your system. Compile *testcurses.vala* (code shown below) with `valac --pkg curses -X -Incurses testcurses.vala` and then run it, to see the system bindings available by default in Vala.

```
using Curses;

int main(string[] args) {
    initscr();
    start_color();
}
```

**10 Years
of Success,
3650 Days
of Growth !**

Special
10th Anniversary
OFFER for
RHCSS
call 9447234635

IPSR is celebrating its
10th Year of operations

**RHCE
RHCVA
RHCSS
RHCDS
RHCA**

Modules starting every week

Join The **WINNING TEAM @ IPSR**

Awards

- Winner of Red Hat Asia Pacific International Award 08 & 09
- Winner of Red Hat GLS National Awards 04, 05, 06, 07 & 08



Faculty & Infrastructure

- 15 member faculty crew led by RHCE, RHCDS, RHCSS certified professionals
- Global standard infrastructure offered at our International Centre located in Kochi

Achievements

- Produced 2500+ RHCEs & 20+ RHCSSs as of April 2010
- 95-100% pass rate for RHCE exams

Placement Support

- All out students are placed through ipsr career centre within 3 months of earning their certification

Management

- Public limited IT company with an exclusive Open Source Based Development Centre (OSBDC)
- Headed by Prof. Dr. Mendus Jacob, who is the Director MCA programme, Marian College, Kuttikanam & Former Director, School of Applicable Mathematics, MG University



ipsr solutions ltd
redefining excellence

Corporate Office : Kottayam
Branches: Kochi North, Kochi South, Kozhikode
Tel : 0484 - 2344560, 0481 - 2561410 / 20
Mob.9447234635, 9447169776
www.ipsr.org • training@ipsrsolutions.com